LESSON 00 · MISSION GUIDE

Day One at Brightwell

Set up your desk, meet the team, and fix your first piece of Dave's code.

From **Gerald Brightwell, Managing Director**
To **Jen, People Analytics Lead**
Subject **Welcome aboard! (and a small favour)**

Jen, welcome to Brightwell Umbrellas! Keeping Britain dry since 1952.

A small confession. Our entire payroll runs on **MACRO-9000**, about 4,000 lines of VBA written by Dave, who left in 2019 to live on a narrowboat. It crashes most Mondays. Nobody dares touch it. The gender pay gap report is due in April, and everyone keeps telling me "Python" is the answer.

I don't know what a Python is, but I've approved the budget (it's free, which I like). Today: get your computer set up and say hello. Tea is in the kitchen. Don't let Biscuit near the keyboard. — G.

Meet the team

Gerald, Managing Director. Grandson of the founder. Sends requests at 16:55 on Fridays.

Mo, IT. Has seen things. Speaks fluent "have you tried turning it off and on again".

Dave, left in 2019. His leftover code turns up in every lesson, always broken.

Priya, Payroll. Sharp, busy, and the source of most of your tickets.

Siobhan, Reward & Benefits. Needs yes/no answers, not "it depends".

Biscuit, office beagle. Not allowed near the keyboard. Often near the keyboard.

What you'll be able to do after today

- [ ] Install Python, VS Code and Git on your own computer
- [ ] Run Python three ways: the interactive prompt, a script, and the ▶ button
- [ ] Explain **repository, commit, branch** and **pull request** in your own words
- [ ] Write a small script, run it, and check it against the expected output
- [ ] Read an error message and find **which line** is broken and **what kind** of problem it is

Why this matters at work

- **VBA lives inside a workbook. Python lives outside.** A Python script can open 300 payroll files overnight without anyone clicking "Enable macros".
- **Git is Track Changes for code**, with a full audit trail: who changed what, when, and why. HR likes audit trails.
- **A script does the same thing every time**, unlike copy-paste at 17:30 on a Friday.

How today works

BEFORE	15 MIN	45 MIN	15 MIN
Setup	**Lecture**	**Exercise sheet**	**Dave's Leftovers**
Follow `SETUP.md`, 30–45 min	Lars shows how it all fits together	Write it all yourself, on your own	Fix his welcome script

Your three documents

GUIDE.pdf	This one. What's going on and why.
EXERCISES.pdf	The exercise sheet. Five exercises; you write every file yourself in `my_work/`.
CHEATSHEET.pdf	One page of commands. Keep it open while you work.

How to check your work

There's no marking robot. Every exercise shows the **expected output**. Run your file and compare: same output means done. If you get an error instead, read it like this:

```
PS> python my_work/hello.py
  File "my_work/hello.py", line 2                  ← WHERE: file and line
    print("A salary of £32,500 a year is", 32500 / 12, "a month"
          ^
SyntaxError: '(' was never closed                  ← WHAT went wrong
```

Read the **last line** first (what went wrong), then go to the **line number** (where). An error message isn't a telling-off. It's Python pointing at the problem for you.

You're done when...

- [ ] `my_work/` contains `hello.py` and `about_me.py`, and both print the expected output
- [ ] Dave's script runs and prints the three expected lines
- [ ] Everything is pushed to `lesson-00` and you've opened a pull request

---



Next lesson: Priya's Inbox. Seven tickets, one crashed macro, and Gerald wants the pay gap "in a sentence". You'll learn how Python handles numbers, text and formatting, the bread and butter of every payroll.